From Retrieve-Then-Read to Learned Search Policies: A Survey of Agentic Retrieval-Augmented Generation

Swapnali Gaikwad[1], Ajit Paraskar[1], Prathamesh Kumbhar[1*], Harshvardhan Patil[1], Bikram Ghosh[1]

[1]Department of Computer Engineering, Dr. D. Y. Patil Institute of Technology, Pimpri, Pune, 411018, Maharashtra, India.

*Corresponding author(s). E-mail(s): prathameshkumbhar.dev@gmail.com;
Contributing authors: swapnali.gaikwad@dypvp.edu.in; ajitparaskar87@gmail.com; harsh.patil.tech@gmail.com; ghoshbikram2725@gmail.com;

Abstract

Retrieval-augmented generation (RAG) lets a large language model answer from documents it was never trained on. The classic recipe retrieves once and then reads, and it breaks down on questions that need several hops, on noisy or misleading passages, and on data that changes. Agentic RAG hands the model control over retrieval: it may split the question, search more than once, choose among tools, decide how much text to read, check its own answer and remember what it learned. This survey reviews that line of work. We cast agentic RAG as a sequential decision problem and use the decisions it exposes to organise the literature into eight dimensions: query planning, iterative retrieval, search policy and evidence granularity, self-verification, tool use, long-term memory, attribution, and evaluation. We compare 25 representative methods along these dimensions, examine in detail the GRASP policy, which learns at each step whether to search semantically, search by keyword or read a whole paragraph, and review benchmarks and attacks specific to retrieval agents. Three patterns stand out. Reinforcement-learned search policies are trained and tested almost entirely on static Wikipedia corpora with gold evidence; verification, memory and actions are studied apart from learned retrieval; and no public benchmark scores an agent that answers from private, changing, multi-source data and also acts on it. We close with open problems that follow from these gaps.

Table 1 Related surveys and what each covers. A check means the topic is treated in some depth; a dash means it is absent or mentioned only in passing.

Survey	RAG	Agents	RL search	Granularity	Security
Gao *et al.* [3]	✓	–	–	–	–
Fan *et al.* [4]	✓	–	–	–	–
Wang *et al.* [17]	–	✓	–	–	–
Singh *et al.* [10]	✓	✓	–	–	–
This survey	✓	✓	✓	✓	✓

Keywords: Retrieval-augmented generation, Agentic RAG, Large language models, Search policy, Citation verification, Agent memory

1 Introduction

A language model knows what was in its training data and nothing after it. Retrieval-augmented generation (RAG) fixes part of this by fetching passages from an external collection and conditioning the answer on them [1, 2]. In its common form a retriever ranks chunks once and a generator reads the top few [3, 4]. The approach works well when one passage holds the answer and the collection does not change. It works badly in three other situations. Some questions can only be searched in steps, because the second half names an entity that the first half has to find [5, 6]. Retrieved text can be wrong or off-topic, and models are easily distracted by it [7, 8]. And some facts, such as deadlines and prices, keep changing [9].

Agentic RAG moves these decisions into the model. The model chooses whether to decompose the question, when to search and with which tool, how much of a result to read, whether its evidence is enough, and what to keep for later [10]. The first systems did this through prompting [5, 11, 12]. Since early 2025 a growing set of methods trains the search behaviour with reinforcement learning (RL), rewarding only the final answer or a few auxiliary signals [13–16].

Several surveys already cover parts of this ground. Gao *et al.* trace RAG from naive through advanced to modular pipelines [3]. Fan *et al.* organise retrieval-augmented LLMs by architecture, training strategy and application [4]. Singh *et al.* classify agentic RAG by agent architecture [10], and Wang *et al.* survey LLM agents in general [17]. Table 1 sets these against the present work. None of them covers the 2025–2026 wave of RL-trained search policies, and none treats evidence granularity, that is, how much text an agent chooses to read, as a decision in its own right.

Contributions. This survey makes four contributions.

1. It frames agentic RAG as a sequential decision problem and derives an eight-dimension taxonomy from the decisions that problem exposes (Sections 4 and 5).
2. It reviews each dimension, including the RL-trained search policies of 2025–2026, and analyses one of them, GRASP [18], in depth, because its ablations isolate the effect of letting the agent choose how much to read (Section 5).

3. It compares 25 representative methods along the taxonomy and reports which combinations of decisions the literature has and has not explored (Section 6), and it reviews benchmarks (Section 7) and attacks specific to retrieval agents (Section 8).
4. It derives open problems from these gaps (Section 9).

2 Review Methodology

Research questions. Three questions guided the review. *RQ1:* Which decisions do agentic RAG systems give to the model, and how can the systems be organised by those decisions? *RQ2:* How do representative methods combine these decisions, and which combinations are missing? *RQ3:* What is still unsolved when agentic RAG leaves static benchmark corpora for private, changing, multi-source data and for tasks with side effects?

Sources and search. We searched arXiv (cs.CL, cs.AI and cs.IR), the ACL Anthology, the NeurIPS, ICLR and ICML proceedings, and Google Scholar for work published from 2018 onwards. Queries combined the terms "retrieval-augmented generation", "agentic RAG", "multi-hop retrieval", "search agent", "reinforcement learning" with "search", "tool use", "agent memory", "attribution" and "prompt injection". We then followed references backwards and citations forwards from the papers we included. The searches were last run in November 2026.

Inclusion and exclusion. We included a work if it (a) lets a language model decide when, how or what to retrieve, or (b) contributes a component that retrieval agents depend on in practice: query reformulation, tool and connector interfaces, long-term memory, claim verification, evaluation, or security. Foundational retrieval and RAG papers were included where later sections build on them. We excluded work that only improves a retriever or reader without changing who controls retrieval, and we kept one version of each paper, preferring the peer-reviewed one. Bibliographic details for every included paper were checked against arXiv and the publisher's proceedings.

Analysis. Each method was read and coded against the eight dimensions of Fig. 2, using only what its own publication describes. GRASP [18] was chosen for close reading because, of the included works, its ablations give the most direct evidence that letting the agent choose the evidence unit changes answer accuracy.

3 Background

3.1 Retrievers

Lexical retrieval with BM25 scores documents by weighted term overlap and favours rare terms [19], so it is strong on names, identifiers and exact phrases. Dense retrievers such as DPR encode questions and passages into one vector space and match by similarity [20]; they catch paraphrases that share no words. Contriever showed that a dense retriever trained without labels can beat BM25 on 11 of 15 BEIR datasets by Recall@100 [21]. Two families sit between these extremes. ColBERT keeps one vector per token and scores a document by late interaction between the token sets [22], and SPLADE learns sparse, expanded term weights that still run on an inverted index [23]. The BEIR benchmark is a useful reality check: across 18 datasets, BM25 remained a

strong baseline out of domain, and re-ranking and late-interaction models were best on average but at a high computational cost [24]. For agentic systems the lesson is that no single signal wins everywhere, which is one motivation for letting the agent choose.

3.2 Retrieval-augmented language models

REALM pre-trained a language model together with a latent retriever, back-propagating through retrieval with masked language modelling as the signal [2]. RAG combined a dense retriever with a sequence-to-sequence generator for knowledge-intensive tasks [1]. Fusion-in-Decoder encodes each retrieved passage separately and fuses them in the decoder, and its accuracy kept rising as more passages were added [25]. RETRO retrieved from a two-trillion-token database and matched GPT-3 on the Pile with 25 times fewer parameters [26], and Atlas reached over 42% on Natural Questions from 64 training examples [27]. Later work showed that architecture changes are not required. In-Context RALM simply prepends retrieved documents to a frozen model's input [28], and REPLUG does the same with a black-box model while using that model's own predictions to train the retriever [29]. Almost every agentic method in this survey builds on this in-context style, because it lets the model's prompt, not its weights, carry the evidence.

3.3 Retrieval units

The unit that is indexed matters as much as the retriever. Indexing short propositions instead of passages improved both retrieval and downstream QA at a fixed reading budget in Dense X [30]. LongRAG goes the other way: it groups Wikipedia into much longer units so that a "long reader" does more of the work and the retriever searches a far smaller collection [31]. RAPTOR builds a tree of recursive cluster summaries for questions that need a broad view [32], and graph-based methods index entities and relations: GraphRAG summarises communities of an entity graph [33], HippoRAG runs Personalized PageRank over a knowledge graph extracted by an LLM [34], and LightRAG combines graph structure with vector retrieval at two levels of detail [35]. In all of these the unit is fixed when the index is built. Section 5.3.1 looks at what happens when the agent chooses it instead.

3.4 Where single-pass RAG fails

More retrieved text is not always better. Adding irrelevant sentences lowers accuracy [7], and models make poor use of information placed in the middle of a long context [8]. RGB breaks the problem into four abilities, noise robustness, negative rejection, information integration and counterfactual robustness, and finds models weak on all of them [36]. Multi-hop datasets such as HotpotQA [37], 2WikiMultiHopQA [38] and MuSiQue [39] hide the second-hop entity until the first hop is solved, so one retrieval round cannot find it; Press *et al.* call the resulting shortfall the "compositionality gap" [6]. A natural objection is that long-context models make retrieval unnecessary. The evidence is mixed. Xu *et al.* found that a 4K-context model with simple retrieval matched a model fine-tuned to 16K context on long-context tasks, at much lower cost, and that retrieval helped at every context length they tried [40]. Li

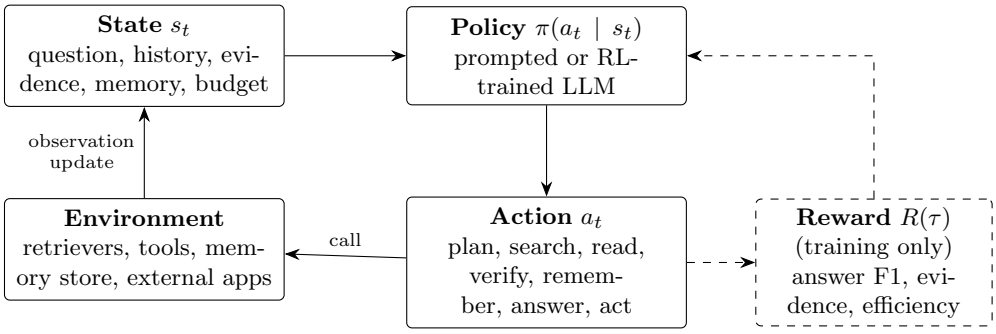


Fig. 1 The decision loop shared by agentic RAG methods. Each design dimension of Fig. 2 corresponds to a part of this loop that a method hands to the model: which actions exist, how the policy is obtained, what the state keeps, and what the reward measures.

et al. found the opposite average result with newer models, long context beating RAG when cost is ignored, and proposed Self-Route, which lets the model decide per query whether retrieval is enough [41]. The two studies disagree on the average case, but in both, retrieval costs far less, and Self-Route keeps accuracy comparable to long context by choosing per query. Deciding *how much* to retrieve is therefore worth learning.

4 Agentic RAG as Sequential Decision-Making

Most agentic RAG methods can be described with one loop (Fig. 1). At step t the agent has a state

$$s_t = (q,\ h_t,\ E_t,\ M,\ b_t), \tag{1}$$

made of the question q, the history of its own reasoning and actions h_t, the evidence gathered so far E_t, any long-term memory M, and the remaining budget b_t of steps, tokens or tool calls. A policy $\pi(a_t \mid s_t)$ chooses an action from a set $\mathcal{A}$ that typically includes rewriting or decomposing the question, calling a retriever or another tool with some query, reading a larger span around a hit, writing to memory, answering, and, in assistants, proposing an action in an external system. The environment returns an observation, the state is updated, and the loop stops when the agent answers or the budget runs out. Training-free methods realise π with a prompted LLM; RL methods optimise it for a reward $R(\tau)$ over the whole trajectory τ, usually answer correctness plus optional terms for evidence quality or efficiency.

Seen this way, the methods in this survey differ along a small number of axes: which actions exist (only "search", or also "read more", "verify" and "remember"), how the policy is obtained (prompting, a trained classifier, supervised fine-tuning or RL), what the state carries across turns and across sessions, and what the reward, if any, measures. These axes give the taxonomy in the next section.

5 A Taxonomy of Agentic RAG

Fig. 2 groups the literature into eight dimensions. A single system usually makes choices along several of them at once; the figure lists each method where its main contribution lies.

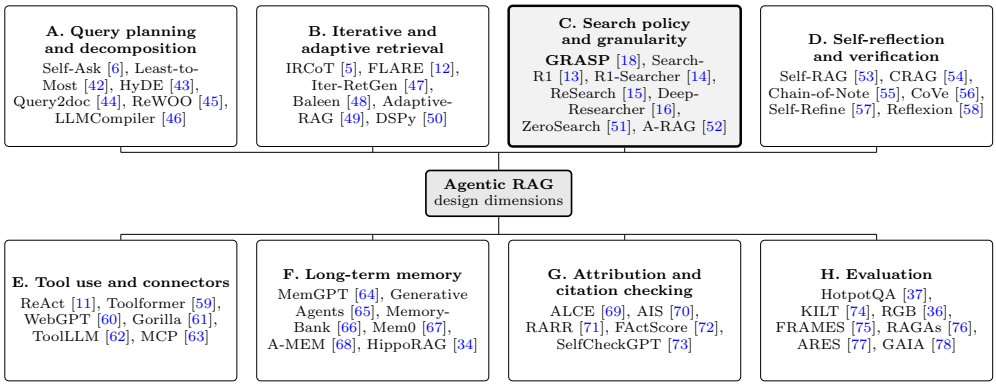


Fig. 2 The eight design dimensions used in this survey, with representative (not exhaustive) methods for each. Dimension C, shaded, holds the RL-trained search policies, including GRASP [18], which Section 5.3.1 examines in detail.

5.1 Query Planning and Decomposition

A question that needs several facts can be broken up before any search runs. Chain-of-thought prompting showed that writing out intermediate steps improves multi-step reasoning [79], and least-to-most prompting made the split explicit: solve easy subproblems first and feed their answers into harder ones [42]. Self-Ask applies this to retrieval, generating follow-up questions and sending each to a search engine [6].

A second group rewrites the query rather than splitting it. HyDE has an LLM write a hypothetical answer document and retrieves real documents near its embedding [43]. Query2doc appends such a pseudo-document to the query and reports gains of 3–15% for BM25 on MS MARCO and TREC DL without any fine-tuning [44]. Rewrite-Retrieve-Read trains a small rewriter with RL, using the reader's success as the reward [80].

The third group concerns *when* the plan is made. ReAct decides each action only after seeing the previous observation [11]. ReWOO separates planning from observation: it writes the whole plan with placeholders for tool outputs, runs the tools, and only then reasons over the results, which cuts token use substantially [45]. LLMCompiler goes further and compiles the plan into a dependency graph whose independent calls run in parallel [46]. The same trade-off appears in all three papers. A plan written up front uses fewer tokens and its calls can run in parallel. A plan built step by step can react to what it finds, and multi-hop questions need that, because the next query depends on the last answer.

5.2 Iterative and Adaptive Retrieval

Iterative methods retrieve during generation rather than before it. IRCoT uses each new chain-of-thought sentence as the next query, so retrieval follows the reasoning [5]. FLARE drafts the next sentence and retrieves only if the draft contains low-confidence tokens [12]. Iter-RetGen alternates between retrieval and generation, using the whole draft answer as the next query [47]. Baleen, an earlier multi-hop retriever, summarises

what each hop found into one condensed context before the next hop, which keeps the search space under control across many hops [48].

Adaptive methods decide *whether* and *how much* to retrieve. Adaptive-RAG trains a small classifier that sends each question to no retrieval, single-step retrieval or multi-step retrieval according to its predicted complexity [49], and Self-Route makes the same kind of choice between RAG and a long context [41]. DSPy treats such pipelines as programs: modules declare what they do, and a compiler optimises prompts and demonstrations for a target metric instead of a human tuning them by hand [50]. All of these vary when retrieval happens or which pipeline runs; the retriever and the size of the text unit stay fixed.

5.3 Learned Search Policies and Evidence Granularity

The most active part of the field since 2025 trains the search loop itself with RL, encouraged by the finding that RL alone can teach long chains of reasoning [81] and by group-relative policy optimisation (GRPO), which removes the need for a separate value model [82]. Search-R1 trains a model to issue search calls inside its reasoning and rewards only the final answer [13]. R1-Searcher uses a two-stage, outcome-based RL scheme and needs neither process rewards nor a distilled cold start [14]. ReSearch treats search as part of the reasoning chain, trains Qwen2.5 models of 7B and 32B parameters on a single dataset, and reports that the learned behaviour transfers to other benchmarks [15]. Search-o1 adds agentic search to large reasoning models at inference time [83].

These methods train against a fixed local corpus. Two papers question that setting from opposite sides. DeepResearcher trains in the real web, arguing that a fixed corpus that is known to contain the answer hides the noise and dead ends of real search [16]. ZeroSearch argues that live search is too costly and too erratic for RL, which needs very many rollouts, and replaces the search engine during training with an LLM that simulates one [51]. Both are trying to make the training environment look more like deployment without paying deployment's price.

A third set of methods widens the action set. MARAG-R1 gives an agent semantic, keyword, filter and aggregation tools and trains it with supervised fine-tuning followed by RL [84]. A-RAG exposes keyword search, semantic search and chunk reading as separate, hierarchical interfaces that a model calls directly [52]. GRASP adds the one action the others lack: the choice of how much to read [18].

5.3.1 GRASP in depth

Problem.

Gandhi *et al.* note that RL-trained retrieval agents are usually tied to one retriever and to fixed chunks of moderate size [18]. The kind and amount of evidence cannot then change from one step to the next. GRASP (GRanularity-Aware Search Policy) learns three decisions in one policy: how to mix lexical and dense search, how much to read, and when to stop.

Table 2 Results reported by GRASP [18]: retrieval recall (R) and answer F1 on 500 validation questions per dataset. GRASP has the best F1 on all three, while IRCoT leads on HotpotQA recall. Numbers are copied from the GRASP paper, not re-run.

	HotpotQA		2Wiki		MuSiQue	
Method	R	F1	R	F1	R	F1
Single-step hybrid	0.86	0.48	0.60	0.32	0.59	0.20
IRCoT	**0.91**	0.36	0.83	0.40	0.62	0.15
Search-R1 (PPO)	0.76	0.47	0.74	0.48	0.55	0.26
Search-R1 (GRPO)	0.74	0.56	0.77	0.53	0.62	0.30
Base (no RL)	0.36	0.24	0.27	0.20	0.24	0.10
GRASP	0.90	**0.66**	**0.90**	**0.60**	**0.70**	**0.33**

Formulation and actions.

The search loop is a Markov decision process with a fixed turn limit. The state holds the question, the history, the evidence and tools used so far, and the remaining budget. Every paragraph is split into sentences that keep a pointer to it. At each step the policy reasons and then takes one action: *semantic search* (Qwen3-0.6B embeddings), *keyword search* (BM25), *read paragraph* (the parent of a retrieved sentence) or *answer*. Searches return the top $K = 5$ sentences, so a full paragraph costs context only when the agent asks for it.

Reward.

The trajectory reward is

$$R = R_A + \alpha R_R + \beta R_S + \gamma R_E, \qquad (\alpha, \beta, \gamma) = (0.7,\, 0.15,\, 0.15). \tag{2}$$

R_A is the token-level F1 of the answer. R_R is the F1 between the paragraphs the agent read and the gold supporting paragraphs, which penalises reading distractors. R_S is 1 only if semantic and keyword search *each* found a gold paragraph, which pushes the agent to use both. $R_E = (T - T_{\text{cur}})/T$, with T the turn budget, rewards finishing early, but it is paid only when $R_A > 0.5$, so a fast wrong answer earns nothing. The auxiliary weights sum to one, which keeps answer F1 dominant and limits reward hacking.

Training and evaluation.

Qwen2.5-3B-Instruct is trained with GRPO [82] on the HotpotQA distractor split [37] for 270 steps, about 19% of one epoch. Episodes end after 10 turns, and retrieved tokens are masked out of the loss. The test sets are 500 validation questions from each of HotpotQA, 2WikiMultiHopQA and MuSiQue [37–39]; the last two are never seen in training. Baselines include single-step lexical, semantic and hybrid retrieval, IRCoT driven by a proprietary model, the public Search-R1 checkpoints trained with PPO and GRPO, and an untrained "Base" agent with GRASP's prompt and tools.

Findings.

GRASP has the best F1 on all three datasets and the best recall on 2WikiMultiHopQA and MuSiQue (Table 2). The authors attribute IRCoT's slightly higher HotpotQA recall to the strong proprietary model that writes its reasoning. Even single-step retrieval beats the untrained Base agent, so tools do little for a model that has not learned to use them. The ablation is the most informative result. On HotpotQA, removing keyword search lowers F1 only from 0.631 to 0.621 and removing semantic search lowers it to 0.567, but removing the read action, so that searches return whole paragraphs, drops it to 0.484. The authors' explanation is that a paragraph shows many entities at once, and the agent's next queries drift back towards the original question instead of following a single clue. A typical learned trajectory starts with a semantic search, reads one paragraph to pick up a bridge entity, runs a keyword search on that entity, and falls back to a broader semantic query if the keyword search fails.

Limitations.

R_R and R_S need gold supporting facts, which few real corpora have. The policy sometimes answers too early, tool use levels off at about eight calls, and only one 3B backbone was tested. The authors propose larger tool sets, rewards built from weaker or model-based grounding signals, and units that range from spans to whole document regions. Their ethics statement recommends provenance, citations, audit logs and human approval for consequential actions, points that return in Section 8.

What GRASP adds to the picture.

Search-R1, R1-Searcher and ReSearch learn *when* and *what* to search. GRASP also learns *how much to read*, and on HotpotQA its ablation finds that removing this choice costs more F1 than removing either retriever. That is the same tension Dense X and LongRAG resolve in opposite directions at index time [30, 31]; GRASP resolves it per step.

5.4 Self-Reflection, Correction and Verification

A second family checks evidence or answers rather than searching better. Self-RAG fine-tunes a model to emit reflection tokens that decide whether to retrieve and grade each passage for relevance and support [53]. CRAG scores what was retrieved with a lightweight evaluator and falls back to web search when the documents look poor [54]. Chain-of-Note has the model write a short reading note on each retrieved document before answering, and to answer "unknown" when neither the documents nor its own knowledge suffice [55].

Other methods correct the answer after it is written. Self-Refine has one model draft, critique and revise its own output, with no training [57]. Chain-of-Verification (CoVe) drafts an answer, plans verification questions about it, answers those questions independently so that the draft cannot bias them, and then writes a revised answer [56]. Reflexion stores written lessons from failed attempts for the next try [58]. GRASP has no critic at inference time; its grounded-reading reward plays that role, but only during training. An open question this raises is whether a verifier at inference

time and a grounding term in the reward are substitutes or complements; we found no study that tests both on the same policy.

5.5 Tool Use and Connector Protocols

Retrieval is one tool among many once a model can call functions. WebGPT trained GPT-3 to browse with a text-based browser, collecting references as it went; its answers were preferred to those of the human demonstrators 56% of the time [60]. ReAct made interleaved reasoning and tool calls the default pattern [11]. Toolformer taught a model from a few demonstrations where an API call helps [59], Gorilla fine-tuned a model to write accurate API calls and paired it with a retriever over API documentation so it can follow documentation changes at test time [61], and ToolLLM built ToolBench from 16,464 real REST APIs in 49 categories [62].

The Model Context Protocol (MCP), released by Anthropic in November 2024, standardises how clients discover and call external tools and data sources [63]. It makes it much easier to connect an agent to private systems such as mail, calendars and code hosts, and each new connection is also a new point of attack [85]; Section 8 returns to this.

5.6 Long-Term Memory

Retrieval over a fixed corpus answers "what does the collection say"; memory answers "what has this user told me before". MemGPT pages information between the context window and external storage much as an operating system pages memory [64]. Generative Agents score each memory by recency, importance and relevance and periodically write higher-level reflections [65]. MemoryBank updates and forgets memories along the Ebbinghaus forgetting curve [66]. Mem0 extracts and consolidates salient facts from conversations, with an optional graph variant [67], and A-MEM organises memories as linked notes in the style of the Zettelkasten method, letting new memories update old ones [68]. HippoRAG, discussed above as a retriever, is also framed by its authors as long-term memory that integrates new experience after pre-training [34].

Evaluation lags behind. LongMemEval tests five abilities, information extraction, multi-session reasoning, temporal reasoning, knowledge updates and abstention, on 500 questions, and reports a 30% accuracy drop for commercial assistants and long-context models over sustained interactions [86]. Once memory holds personal data, its provenance, expiry and deletion become design requirements rather than afterthoughts, and none of the benchmarks above measures them.

5.7 Attribution and Citation Checking

An answer that cites its sources can still misrepresent them. ALCE measures whether cited passages actually support each statement and finds that, often, they do not [69]. AIS defines attribution to identified sources as a rating task for humans [70]. FActScore splits a generation into atomic facts and scores each against a knowledge source [72]. RARR works after the fact: it finds evidence for a model's output and post-edits unsupported content while keeping the rest [71]. SelfCheckGPT needs no external source at all, flagging statements that vary across several sampled answers as likely

Table 3 Representative methods along the taxonomy dimensions, as described in each publication. Reading down the columns shows which decisions are rarely combined: no RL-trained search policy also keeps long-term memory or takes actions in external systems.

Method	Retrieval strategy	Granularity	Policy	Memory	Verification	Live data	Actions
RAG [1]	Single-shot dense retrieval	Fixed passage	Fine-tuned retriever, generator	–	–	–	–
REPLUG [29]	Prepend to black-box LM	Fixed passage	LM-supervised retriever	–	–	–	–
Self-Ask [6]	Follow-up sub-questions	Search-engine answer	Prompting	–	–	Web search	–
ReWOO [45]	Plan first, then call tools	Tool output	Prompting	–	–	Tools	–
LLMCompiler [46]	Parallel tool-call graph	Tool output	Prompting (planner)	–	–	Tools	Function calls
IRCoT [5]	Retrieve at each reasoning step	Fixed paragraph	Prompting	–	–	–	–
FLARE [12]	Retrieve on low confidence	Fixed passage	Prompting	–	Regenerate uncertain sentence	–	–
Adaptive-RAG [49]	None, single or multi-step	Fixed	Trained classifier	–	–	–	–
HippoRAG [34]	PageRank over LLM-built graph	Passage via graph	Fixed algorithm	Graph index	–	–	–
Self-RAG [53]	On demand, reflection tokens	Fixed passage	Fine-tuned	–	Relevance, support tokens	–	–
CRAG [54]	Evaluator-triggered correction	Strip filtering	Trained evaluator	–	Retrieval evaluator	Web fallback	–
Chain-of-Note [55]	Notes on each document	Fixed passage	Fine-tuned	–	Reading notes, "unknown"	–	–
CoVe [56]	Answer, then verify	–	Prompting	–	Independent verification questions	–	–
ReAct [11]	Interleaved thought and action	API page or lookup	Prompting	–	–	Wikipedia API	Environment actions
WebGPT [60]	Text-based browsing	Quoted spans	Imitation, then RLHF	–	Collected references	Live web	Browser commands
Search-R1 [13]	Search inside reasoning	Fixed passage	RL (PPO, GRPO)	–	–	–	–
R1-Searcher [14]	Search inside reasoning	Fixed passage	Two-stage outcome RL	–	–	–	–
ReSearch [15]	Search as reasoning step	Fixed passage	RL, no step labels	–	–	–	–
DeepResearcher [16]	Multi-step web research	Web page	RL in the real web	–	–	Live web	Browsing
ZeroSearch [51]	Search inside reasoning	Simulated documents	RL with simulated search	–	–	–	–
MARAG-R1 [84]	Semantic, keyword, filter, aggregate	Fixed chunk	SFT, then RL	–	–	–	–
A-RAG [52]	Hierarchical retrieval interfaces	Keyword to chunk	Model-driven tool use	–	–	–	–
GRASP [18]	Coordinated semantic and keyword	Sentence; paragraph on demand	RL (GRPO), 4-part reward	–	Grounded reading (training only)	–	–
MemGPT [64]	Paging context and storage	–	Function calls	Tiered context	–	–	–
A-MEM [68]	Linked-note retrieval	Note	Prompting	Evolving note graph	–	–	–

hallucinations [73]. These methods score individual claims, which makes them a natural source of the weaker, model-based grounding signal that GRASP lists as future work; Section 9 discusses the risk in that idea.

6 Comparative Analysis

Table 3 codes 25 representative methods along the taxonomy. "Granularity" records whether the evidence unit is fixed in advance or chosen by the agent, and "Live data" whether the sources can change while the agent runs. A dash means the method does not address that dimension in its publication.

Four patterns follow from the table.

Learned policies live in static worlds. Every RL-trained search policy except DeepResearcher is trained and tested on a fixed, mostly Wikipedia-based corpus in which the answer is known to exist, and the strongest auxiliary rewards (GRASP's R_R and R_S) need gold supporting facts. Methods that query live sources (Self-Ask, ReAct, WebGPT, CRAG's fallback) mostly rely on prompting. ZeroSearch's simulated search engine and DeepResearcher's real one are the two attempts we found to close this gap, and they pull in opposite directions on cost and realism.

Granularity is rarely a decision. Only A-RAG and GRASP let the agent choose the evidence unit at query time. Index-time choices (propositions, long units, summary trees, graphs) are common, and the evidence from Dense X, LongRAG and GRASP's ablation together suggests the right unit depends on the question, which is an argument for making it a per-step choice.

Verification and retrieval are studied apart. Methods with explicit verification (Self-RAG, CRAG, Chain-of-Note, CoVe) use fixed retrieval; methods with learned retrieval have no verifier at inference time. The two families answer different halves of the same problem, finding evidence and judging it, and the table contains no method that learns both.

Memory and actions are absent from learned retrieval. Long-term memory appears only in memory-specific systems, and side-effecting actions only in prompted tool agents and in DeepResearcher's browsing. No method in the table combines an RL-trained search policy with persistent memory or with actions in external systems, although deployed assistants need all three.

7 Benchmarks and Evaluation

Table 4 summarises the benchmarks used by the surveyed work and what each one tests.

Most agentic RAG work, GRASP included, reports results on the three Wikipedia multi-hop datasets. They are well understood and provide gold evidence, which RL rewards need, but they share three properties that real deployments lack: one clean source, facts that do not change, and no actions. Newer benchmarks address one property each. MultiHop-RAG moves to news [87]; FreshQA targets facts that change [9]; FRAMES evaluates retrieval and reasoning together instead of in isolation [75]; GAIA and BrowseComp score agents that browse and use tools [78, 88]. Reference-free scorers such as RAGAs and ARES make evaluation cheaper; ARES fine-tunes small judges on synthetic data and corrects them with a few hundred human labels [76, 77].

What is missing is a benchmark that combines the properties. We found none in which an agent answers from private, heterogeneous sources (mail, chat, documents, code), those sources change during the evaluation, answers must cite and be checked against evidence, honest abstention counts as success, conflicts between sources must be surfaced, and some tasks require an action whose safety is scored. Each of these appears somewhere in Table 4, but never together.

Table 4 Benchmarks for retrieval agents and what each one measures. Sizes are as reported by the benchmark authors where they state one.

Benchmark	Setting	What it tests
HotpotQA, 2Wiki, MuSiQue [37–39]	Wikipedia, multi-hop	Bridge and comparison questions; gold supporting facts
KILT [74]	One Wikipedia snapshot	Several knowledge tasks; provenance of answers
BEIR [24]	18 retrieval datasets	Zero-shot retrieval across domains
RGB [36]	English and Chinese	Noise, negative rejection, integration, counterfactuals
MultiHop-RAG [87]	News articles	Multi-hop queries over a realistic knowledge base
FRAMES [75]	824 questions	Factuality, retrieval and reasoning together
FreshQA [9]	Changing facts	Answers whose truth changes over time
RAGAs, ARES [76, 77]	Any RAG system	Faithfulness and relevance without full references
GAIA [78]	Assistant tasks	Reasoning, browsing, tool use (humans 92%, GPT-4 with plugins 15%)
BrowseComp [88]	1,266 web questions	Persistent search for hard-to-find facts
AgentBench [89]	8 environments	General LLM-as-agent decision-making
LongMemEval [86]	500 questions	Five long-term memory abilities in chat

8 Security and Robustness

Giving a model control over retrieval also gives an attacker more ways in. Three threats recur in the surveyed work.

Indirect prompt injection. Text that the agent retrieves can contain instructions, and the model may follow them [90]. The risk grows with each tool an agent can call. InjecAgent measures it with 1,054 test cases spanning 17 user tools and 62 attacker tools, separating direct harm to the user from exfiltration of private data [91]. AgentDojo provides 97 realistic tasks, such as managing an email client or making travel bookings, with 629 security test cases, and is built as an extensible environment so that new attacks and defences can be added [92].

Knowledge corruption. PoisonedRAG shows that injecting a few malicious texts into the knowledge base can steer a RAG system to an attacker-chosen answer for a target question [93]. For an agent that searches repeatedly, each search is another chance to retrieve poisoned text.

Connector and protocol risk. Protocols such as MCP make it easy to attach many tools to one agent [63], and every server added is code and data the agent trusts [85].

The defences in the literature are partial. Approval gates, under which consequential actions run only after a human confirms an exact preview, limit the damage an

injected instruction can do; GRASP's authors recommend human approval for consequential actions for this reason [18]. Separating retrieved data from instructions, provenance on every piece of evidence, and audit logs reduce the risk further. None of these removes it, and the AgentDojo authors report that their environment remains a challenge for attacks and defences alike [92].

9 Open Challenges and Future Directions

Supervision without gold evidence. The richest RL rewards need gold supporting facts [18, 84], which private corpora do not have. Claim-level scores of the AIS or FActScore kind [70, 72], verifier labels, and agreement across sampled trajectories [73] are candidate substitutes. Each brings a risk: a policy rewarded by a partly lexical or model-based checker can learn to satisfy the checker rather than the user. Human spot checks, and rewards that combine several weak signals, are the obvious safeguards; we found no study that measures how much reward gaming they prevent.

Evidence units for heterogeneous data. GRASP's sentence-and-paragraph pair has no obvious counterpart in an email thread, a pull request, a calendar event or a chat channel. Hierarchical units [32, 52], propositions [30], long units [31] and graphs [33, 34] give starting points, but the right "read more" action probably differs by source.

Learning retrieval and verification together. The comparison in Section 6 shows the two developing separately. A policy that searches, judges its evidence with a learned critic, and decides to search again when the critic disagrees would join the strengths of the GRASP and Self-RAG families.

Freshness and conflict. Time-sensitive questions remain hard [9], and most methods treat conflicting evidence as noise to filter. When sources disagree, for example two documents giving different deadlines, a useful agent should show the disagreement and its sources rather than pick a side silently.

Memory governance. Memory systems [64, 66–68] raise questions that benchmarks such as LongMemEval do not yet measure: whether the user can see why something was remembered, whether stale memories expire, and whether deletion is complete.

Safety of acting agents. Prompt injection and knowledge corruption remain open [91–93]. Approval gates help, but they move the burden to the user; methods that reduce how often a user must approve, without raising risk, are needed.

Cost-aware evaluation. GRASP settles at about eight tool calls [18], and LLMCompiler and ReWOO show how much latency and token cost depend on the plan [45, 46]. Benchmarks should report cost alongside accuracy, and should credit citation support, honest abstention, conflict reporting and action safety, not answer accuracy alone.

10 Conclusion

Agentic RAG turns retrieval from a fixed preprocessing step into a sequence of decisions made by the model. Framed as a decision problem, the literature splits into eight

dimensions. Each dimension has made progress on its own, but few methods combine them. RL-trained search policies such as Search-R1, R1-Searcher, ReSearch and GRASP now learn when and what to search, and GRASP's ablation on HotpotQA suggests that learning how much to read matters more than which retriever is used. Verification, memory and safe action have their own strong methods. Yet the learned policies are trained on static corpora with gold evidence, none of them verifies at inference time or keeps memory, and no benchmark tests an agent over private, changing data that it can also act on. Closing those gaps needs rewards that do not depend on gold evidence, evidence units suited to mail, chat and code, joint training of retrieval and verification, and evaluations that score trust and safety alongside accuracy.

Threats to Validity

The survey covers work found through the search described in Section 2; relevant systems outside those sources, or published after the search, may be missing. Several recent works, including some of the RL-trained search policies, were available only as preprints at the time of writing, and all reported numbers are reproduced from the original papers rather than re-run. The coding of methods in Table 3 reflects what each publication describes and may understate capabilities that a method has but does not report.

Declarations

- **Funding:** No funding was received for this study.
- **Conflict of interest:** On behalf of all authors, the corresponding author states that there is no conflict of interest.
- **Ethics approval:** Not applicable; this is a literature review with no human participants or data.
- **Data availability:** No new data were generated or analysed. All results discussed are taken from the cited publications.
- **Author contributions:** All authors contributed to the literature search, analysis and writing of the manuscript. S.G. supervised the work. All authors read and approved the final manuscript.
- **Use of generative AI:** An AI assistant was used to help structure the manuscript, to draft and edit sections based on the authors' earlier conference paper and notes, and for reference management. All AI-assisted text was reviewed and revised by the authors, every reference was checked against the original source, and the authors take full responsibility for the content.

References

[1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474 (2020)

[2] Guu, K., Lee, K., Tung, Z., Pasupat, P., Chang, M.-W.: REALM: Retrieval-Augmented language model Pre-Training. In: Proceedings of the 37th International Conference on Machine Learning (ICML) (2020). arXiv:2002.08909

[3] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Guo, Q., Wang, M., Wang, H.: Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv preprint arXiv:2312.10997 (2023)

[4] Fan, W., Ding, Y., Ning, L., Wang, S., Li, H., Yin, D., Chua, T.-S., Li, Q.: A survey on RAG meeting LLMs: Towards Retrieval-Augmented large language models. In: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), pp. 6491–6501 (2024). arXiv:2405.06211

[5] Trivedi, H., Balasubramanian, N., Khot, T., Sabharwal, A.: Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In: Proc. 61st Annu. Meeting of the Association for Computational Linguistics (ACL), pp. 10014–10037 (2023)

[6] Press, O., Zhang, M., Min, S., Schmidt, L., Smith, N.A., Lewis, M.: Measuring and narrowing the compositionality gap in language models. In: Findings of the Association for Computational Linguistics: EMNLP 2023, pp. 5687–5711 (2023)

[7] Shi, F., Chen, X., Misra, K., Scales, N., Dohan, D., Chi, E.H., Schärli, N., Zhou, D.: Large language models can be easily distracted by irrelevant context. In: Proc. 40th Int. Conf. Machine Learning (ICML) (2023)

[8] Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., Liang, P.: Lost in the middle: How language models use long contexts. Transactions of the Association for Computational Linguistics **12**, 157–173 (2024)

[9] Vu, T., Iyyer, M., Wang, X., Constant, N., Wei, J., Wei, J., Tar, C., Sung, Y.-H., Zhou, D., Le, Q., Luong, T.: FreshLLMs: Refreshing large language models with search engine augmentation. In: Findings of the Association for Computational Linguistics: ACL 2024, pp. 13697–13720 (2024)

[10] Singh, A., Ehtesham, A., Kumar, S., Khoei, T.T.: Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG. arXiv preprint arXiv:2501.09136 (2025)

[11] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: ReAct: Synergizing reasoning and acting in language models. In: Proc. Int. Conf. Learning Representations (ICLR) (2023)

[12] Jiang, Z., Xu, F.F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., Yang, Y., Callan, J., Neubig, G.: Active retrieval augmented generation. In: Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), pp. 7969–7992 (2023)

[13] Jin, B., Zeng, H., Yue, Z., Yoon, J., Arik, S., Wang, D., Zamani, H., Han,

J.: Search-R1: Training LLMs to reason and leverage search engines with reinforcement learning. In: Proc. Conf. Language Modeling (COLM) (2025). arXiv:2503.09516

[14] Song, H., Jiang, J., Min, Y., Chen, J., Chen, Z., Zhao, W.X., Fang, L., Wen, J.-R.: R1-Searcher: Incentivizing the Search Capability in LLMs via Reinforcement Learning. arXiv preprint arXiv:2503.05592 (2025)

[15] Chen, M., Sun, L., Li, T., Sun, H., Zhou, Y., Zhu, C., Wang, H., Pan, J.Z., Zhang, W., Chen, H., *et al.*: ReSearch: Learning to reason with search for LLMs via reinforcement learning. In: Advances in Neural Information Processing Systems (NeurIPS) (2025). arXiv:2503.19470

[16] Zheng, Y., Fu, D., Hu, X., Cai, X., Ye, L., Lu, P., Liu, P.: DeepResearcher: Scaling deep research via reinforcement learning in real-world environments. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 414–431 (2025). arXiv:2504.03160

[17] Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., *et al.*: A survey on large language model based autonomous agents. Frontiers of Computer Science **18**(6), 186345 (2024). arXiv:2308.11432

[18] Gandhi, V., Lee, J., Todmal, S., Dernoncourt, F., Rossi, R., Wang, Z., Lan, A.: GRASP: GRanularity-Aware Search Policy for Agentic RAG. arXiv preprint arXiv:2607.10463. Listed by Adobe Research as appearing in Findings of EMNLP 2026 (2026)

[19] Robertson, S., Zaragoza, H.: The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval **3**(4), 333–389 (2009)

[20] Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., Yih, W.-t.: Dense passage retrieval for open-domain question answering. In: Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), pp. 6769–6781 (2020)

[21] Izacard, G., Caron, M., Hosseini, L., Riedel, S., Bojanowski, P., Joulin, A., Grave, E.: Unsupervised dense information retrieval with contrastive learning. Transactions on Machine Learning Research (2022). arXiv:2112.09118

[22] Khattab, O., Zaharia, M.: ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 39–48 (2020). arXiv:2004.12832

[23] Formal, T., Piwowarski, B., Clinchant, S.: SPLADE: Sparse lexical and expansion model for first stage ranking. In: Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval (2021).

arXiv:2107.05720

[24] Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., Gurevych, I.: BEIR: A heterogenous benchmark for zero-shot evaluation of information retrieval models. In: Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track (2021). arXiv:2104.08663

[25] Izacard, G., Grave, E.: Leveraging passage retrieval with generative models for open domain question answering. In: Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL), pp. 874–880 (2021). arXiv:2007.01282

[26] Borgeaud, S., Mensch, A., Hoffmann, J., Cai, T., Rutherford, E., Millican, K., Driessche, G., Lespiau, J.-B., Damoc, B., Clark, A., *et al.*: Improving language models by retrieving from trillions of tokens. In: Proceedings of the 39th International Conference on Machine Learning (ICML) (2022). arXiv:2112.04426

[27] Izacard, G., Lewis, P., Lomeli, M., Hosseini, L., Petroni, F., Schick, T., Dwivedi-Yu, J., Joulin, A., Riedel, S., Grave, E.: Atlas: Few-shot learning with retrieval augmented language models. Journal of Machine Learning Research **24**(251), 1–43 (2023). arXiv:2208.03299

[28] Ram, O., Levine, Y., Dalmedigos, I., Muhlgay, D., Shashua, A., Leyton-Brown, K., Shoham, Y.: In-Context Retrieval-Augmented language models. Transactions of the Association for Computational Linguistics **11** (2023). arXiv:2302.00083

[29] Shi, W., Min, S., Yasunaga, M., Seo, M., James, R., Lewis, M., Zettlemoyer, L., Yih, W.-t.: REPLUG: Retrieval-Augmented Black-Box language models. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL), pp. 8371–8384 (2024). arXiv:2301.12652

[30] Chen, T., Wang, H., Chen, S., Yu, W., Ma, K., Zhao, X., Zhang, H., Yu, D.: Dense X retrieval: What retrieval granularity should we use? In: Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), pp. 15159–15177 (2024)

[31] Jiang, Z., Ma, X., Chen, W.: LongRAG: Enhancing Retrieval-Augmented Generation with Long-context LLMs. arXiv preprint arXiv:2406.15319 (2024)

[32] Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., Manning, C.D.: RAPTOR: Recursive abstractive processing for tree-organized retrieval. In: Proc. Int. Conf. Learning Representations (ICLR) (2024)

[33] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R.O., Larson, J.: From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv preprint arXiv:2404.16130 (2024)

[34] Gutiérrez, B.J., Shu, Y., Gu, Y., Yasunaga, M., Su, Y.: HippoRAG: Neurobiologically inspired Long-Term memory for large language models. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 37 (2024). arXiv:2405.14831

[35] Guo, Z., Xia, L., Yu, Y., Ao, T., Huang, C.: LightRAG: Simple and Fast Retrieval-Augmented Generation. arXiv preprint arXiv:2410.05779 (2024)

[36] Chen, J., Lin, H., Han, X., Sun, L.: Benchmarking large language models in Retrieval-Augmented generation. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38, pp. 17754–17762 (2024). arXiv:2309.01431

[37] Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W.W., Salakhutdinov, R., Manning, C.D.: HotpotQA: A dataset for diverse, explainable multi-hop question answering. In: Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), pp. 2369–2380 (2018)

[38] Ho, X., Nguyen, A.-K.D., Sugawara, S., Aizawa, A.: Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In: Proc. 28th Int. Conf. Computational Linguistics (COLING), pp. 6609–6625 (2020)

[39] Trivedi, H., Balasubramanian, N., Khot, T., Sabharwal, A.: MuSiQue: Multihop questions via single-hop question composition. Transactions of the Association for Computational Linguistics **10**, 539–554 (2022)

[40] Xu, P., Ping, W., Wu, X., McAfee, L., Zhu, C., Liu, Z., Subramanian, S., Bakhturina, E., Shoeybi, M., Catanzaro, B.: Retrieval meets long context large language models. In: International Conference on Learning Representations (ICLR) (2024). arXiv:2310.03025

[41] Li, Z., Li, C., Zhang, M., Mei, Q., Bendersky, M.: Retrieval augmented generation or Long-Context LLMs? a comprehensive study and hybrid approach. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track, pp. 881–893 (2024). arXiv:2407.16833

[42] Zhou, D., Schärli, N., Hou, L., Wei, J., Scales, N., Wang, X., Schuurmans, D., Cui, C., Bousquet, O., Le, Q., Chi, E.: Least-to-Most prompting enables complex reasoning in large language models. In: International Conference on Learning Representations (ICLR) (2023). arXiv:2205.10625

[43] Gao, L., Ma, X., Lin, J., Callan, J.: Precise Zero-Shot dense retrieval without relevance labels. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp. 1762–1777 (2023). arXiv:2212.10496

[44] Wang, L., Yang, N., Wei, F.: Query2doc: Query expansion with large language models. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 9414–9423 (2023). arXiv:2303.07678

[45] Xu, B., Peng, Z., Lei, B., Mukherjee, S., Liu, Y., Xu, D.: ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models. arXiv preprint arXiv:2305.18323 (2023)

[46] Kim, S., Moon, S., Tabrizi, R., Lee, N., Mahoney, M.W., Keutzer, K., Gholami, A.: An LLM compiler for parallel function calling. In: Proceedings of the 41st International Conference on Machine Learning (ICML) (2024). arXiv:2312.04511

[47] Shao, Z., Gong, Y., Shen, Y., Huang, M., Duan, N., Chen, W.: Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy. In: Findings of the Association for Computational Linguistics: EMNLP 2023, pp. 9248–9274 (2023)

[48] Khattab, O., Potts, C., Zaharia, M.: Baleen: Robust Multi-Hop reasoning at scale via condensed retrieval. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 34, pp. 27670–27682 (2021). arXiv:2101.00436

[49] Jeong, S., Baek, J., Cho, S., Hwang, S.J., Park, J.C.: Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In: Proc. Conf. North American Chapter of the Association for Computational Linguistics (NAACL), pp. 7036–7050 (2024)

[50] Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., Haq, S., Sharma, A., Joshi, T.T., Moazam, H., *et al.*: DSPy: Compiling declarative language model calls into Self-Improving pipelines. In: International Conference on Learning Representations (ICLR) (2024). arXiv:2310.03714

[51] Sun, H., Qiao, Z., Guo, J., Fan, X., Hou, Y., Jiang, Y., Xie, P., Zhang, Y., Huang, F., Zhou, J.: ZeroSearch: Incentivize the Search Capability of LLMs without Searching. arXiv preprint arXiv:2505.04588 (2025)

[52] Du, M., Xu, B., Zhu, C., Wang, S., Wang, P., Wang, X., Mao, Z.: A-RAG: Scaling Agentic Retrieval-Augmented Generation via Hierarchical Retrieval Interfaces. arXiv preprint arXiv:2602.03442 (2026)

[53] Asai, A., Wu, Z., Wang, Y., Sil, A., Hajishirzi, H.: Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In: Proc. Int. Conf. Learning Representations (ICLR) (2024)

[54] Yan, S.-Q., Gu, J.-C., Zhu, Y., Ling, Z.-H.: Corrective Retrieval Augmented Generation. arXiv preprint arXiv:2401.15884 (2024)

[55] Yu, W., Zhang, H., Pan, X., Ma, K., Wang, H., Yu, D.: Chain-of-Note: Enhancing robustness in Retrieval-Augmented language models. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 14672–14685 (2024). arXiv:2311.09210

[56] Dhuliawala, S., Komeili, M., Xu, J., Raileanu, R., Li, X., Celikyilmaz, A., Weston, J.: Chain-of-Verification reduces hallucination in large language models. In: Findings of the Association for Computational Linguistics: ACL 2024, pp. 3563–3578 (2024). arXiv:2309.11495

[57] Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., *et al.*: Self-Refine: Iterative refinement with Self-Feedback. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 36 (2023). arXiv:2303.17651

[58] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: Language agents with verbal reinforcement learning. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 36 (2023)

[59] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., Scialom, T.: Toolformer: Language models can teach themselves to use tools. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 36, pp. 68539–68551 (2023)

[60] Nakano, R., Hilton, J., Balaji, S., Wu, J., Ouyang, L., Kim, C., Hesse, C., Jain, S., Kosaraju, V., Saunders, W., et al.: WebGPT: Browser-assisted question-answering with human feedback. arXiv preprint arXiv:2112.09332 (2021)

[61] Patil, S.G., Zhang, T., Wang, X., Gonzalez, J.E.: Gorilla: Large language model connected with massive APIs. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 37 (2024). arXiv:2305.15334

[62] Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., *et al.*: ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In: International Conference on Learning Representations (ICLR) (2024). arXiv:2307.16789

[63] Anthropic: Introducing the Model Context Protocol. https://www.anthropic.com/news/model-context-protocol (2024)

[64] Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S.G., Stoica, I., Gonzalez, J.E.: MemGPT: Towards LLMs as Operating Systems. arXiv preprint arXiv:2310.08560 (2023)

[65] Park, J.S., O'Brien, J.C., Cai, C.J., Morris, M.R., Liang, P., Bernstein, M.S.: Generative agents: Interactive simulacra of human behavior. In: Proc. 36th Annu. ACM Symp. User Interface Software and Technology (UIST) (2023)

[66] Zhong, W., Guo, L., Gao, Q., Ye, H., Wang, Y.: MemoryBank: Enhancing large language models with Long-Term memory. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38 (2024). arXiv:2305.10250

[67] Chhikara, P., Khant, D., Aryan, S., Singh, T., Yadav, D.: Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv preprint arXiv:2504.19413 (2025)

[68] Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., Zhang, Y.: A-MEM: Agentic memory for LLM agents. In: Advances in Neural Information Processing Systems (NeurIPS) (2025). arXiv:2502.12110

[69] Gao, T., Yen, H., Yu, J., Chen, D.: Enabling large language models to generate text with citations. In: Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), pp. 6465–6488 (2023)

[70] Rashkin, H., Nikolaev, V., Lamm, M., Aroyo, L., Collins, M., Das, D., Petrov, S., Tomar, G.S., Turc, I., Reitter, D.: Measuring attribution in natural language generation models. Computational Linguistics **49**(4), 777–840 (2023)

[71] Gao, L., Dai, Z., Pasupat, P., Chen, A., Chaganty, A.T., Fan, Y., Zhao, V.Y., Lao, N., Lee, H., Juan, D.-C., Guu, K.: RARR: Researching and revising what language models say, using language models. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), pp. 16477–16508 (2023). arXiv:2210.08726

[72] Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W.-t., Koh, P.W., Iyyer, M., Zettlemoyer, L., Hajishirzi, H.: FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In: Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), pp. 12076–12100 (2023)

[73] Manakul, P., Liusie, A., Gales, M.J.F.: SelfCheckGPT: Zero-Resource Black-Box hallucination detection for generative large language models. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP) (2023). arXiv:2303.08896

[74] Petroni, F., Piktus, A., Fan, A., Lewis, P., Yazdani, M., Cao, N.D., Thorne, J., Jernite, Y., Karpukhin, V., Maillard, J., *et al.*: KILT: a benchmark for knowledge intensive language tasks. In: Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL), pp. 2523–2544 (2021). arXiv:2009.02252

[75] Krishna, S., Krishna, K., Mohananey, A., Schwarcz, S., Stambler, A., Upadhyay, S., Faruqui, M.: Fact, fetch, and reason: A unified evaluation of Retrieval-Augmented generation. In: Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL), pp. 4745–4759 (2025). arXiv:2409.12941

[76] Es, S., James, J., Espinosa-Anke, L., Schockaert, S.: RAGAs: Automated evaluation of retrieval augmented generation. In: Proc. 18th Conf. European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL),

pp. 150–158 (2024)

[77] Saad-Falcon, J., Khattab, O., Potts, C., Zaharia, M.: ARES: An automated evaluation framework for Retrieval-Augmented generation systems. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL) (2024). arXiv:2311.09476

[78] Mialon, G., Fourrier, C., Swift, C., Wolf, T., LeCun, Y., Scialom, T.: GAIA: a benchmark for general AI assistants. In: International Conference on Learning Representations (ICLR) (2024). arXiv:2311.12983

[79] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., Zhou, D.: Chain-of-Thought prompting elicits reasoning in large language models. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 35 (2022). arXiv:2201.11903

[80] Ma, X., Gong, Y., He, P., Zhao, H., Duan, N.: Query rewriting in retrieval-augmented large language models. In: Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), pp. 5303–5315 (2023)

[81] DeepSeek-AI: DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. Nature **645**, 633–638 (2025). arXiv:2501.12948

[82] Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y.K., Wu, Y., Guo, D.: DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv preprint arXiv:2402.03300 (2024)

[83] Li, X., Dong, G., Jin, J., Zhang, Y., Zhou, Y., Zhu, Y., Zhang, P., Dou, Z.: Search-o1: Agentic Search-Enhanced Large Reasoning Models. arXiv preprint arXiv:2501.05366 (2025)

[84] Luo, Q., Li, X., Wang, Y., Fan, T., Li, Y., Chen, X., Qiu, X.: MARAG-R1: Beyond Single Retriever via Reinforcement-Learned Multi-Tool Agentic Retrieval. arXiv preprint arXiv:2510.27569 (2025)

[85] Hou, X., Zhao, Y., Wang, S., Wang, H.: Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. arXiv preprint arXiv:2503.23278 (2025)

[86] Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K.-W., Yu, D.: LongMemEval: Benchmarking chat assistants on Long-Term interactive memory. In: International Conference on Learning Representations (ICLR) (2025). arXiv:2410.10813

[87] Tang, Y., Yang, Y.: MultiHop-RAG: Benchmarking Retrieval-Augmented generation for Multi-Hop queries. In: Conference on Language Modeling (COLM) (2024). arXiv:2401.15391

[88] Wei, J., Sun, Z., Papay, S., McKinney, S., Han, J., Fulford, I., Chung, H.W., Passos, A.T., Fedus, W., Glaese, A.: BrowseComp: A Simple Yet Challenging Benchmark for Browsing Agents. arXiv preprint arXiv:2504.12516 (2025)

[89] Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., *et al.*: AgentBench: Evaluating LLMs as agents. In: International Conference on Learning Representations (ICLR) (2024). arXiv:2308.03688

[90] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M.: Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In: Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec), pp. 79–90 (2023)

[91] Zhan, Q., Liang, Z., Ying, Z., Kang, D.: InjecAgent: Benchmarking indirect prompt injections in Tool-Integrated large language model agents. In: Findings of the Association for Computational Linguistics: ACL 2024, pp. 10471–10506 (2024). arXiv:2403.02691

[92] Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., Tramèr, F.: AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In: Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, vol. 37 (2024). arXiv:2406.13352

[93] Zou, W., Geng, R., Wang, B., Jia, J.: PoisonedRAG: Knowledge corruption attacks to Retrieval-Augmented generation of large language models. In: Proceedings of the 34th USENIX Security Symposium, pp. 3827–3844 (2025). arXiv:2402.07867